REDOWAN DELOWAR

Context propagation in Go

Lessons from operating large-scale distributed systems at DoorDash (Wolt)



Scan to get the slides

https://rednafi.com/appearances

$ whoami

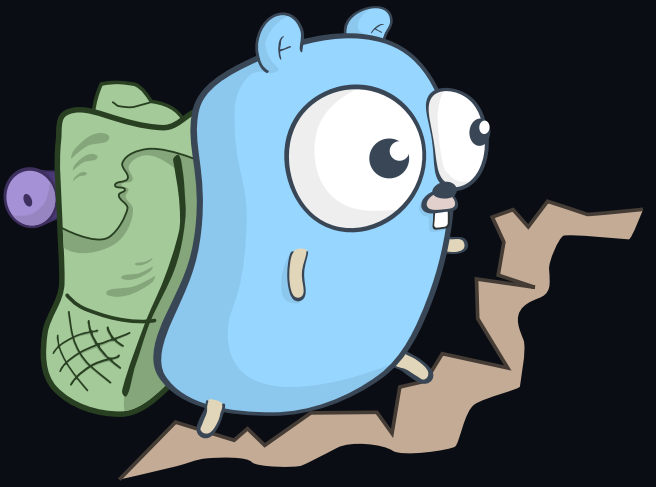
Hi, I'm Redowan, @rednafi on the web.

I oscillate between product and platform work at DoorDash/Wolt.

Interested in databases, streaming, o11y, and failure modes of large-scale distsys.

Writing at rednafi.com.

github.com/rednafi
linkedin.com/in/rednafi



Overview

01 A log line with no cause

02 Context primer

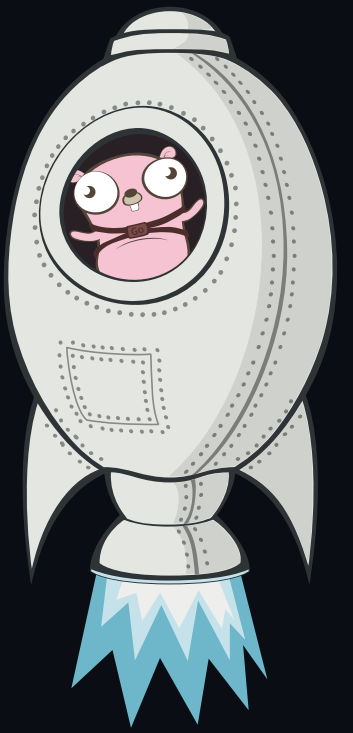
03 Passing a context down

04 Giving every cancel a cause

05 Context values

06 Enforcing the rules

07 Checklist and limits



The log doesn't say why the context stopped

order-service

```
ERROR place order: context canceled
ERROR fetch menu: context deadline exceeded
ERROR place order: context canceled
ERROR charge card: context canceled
ERROR fetch menu: context deadline exceeded
ERROR place order: context canceled
```

ANY OF THESE CAN STOP THE CONTEXT

- The client hung up
- A caller's deadline passed
- The server started to shut down
- Code called `cancel()`

The cancel often comes from another service or from a library.

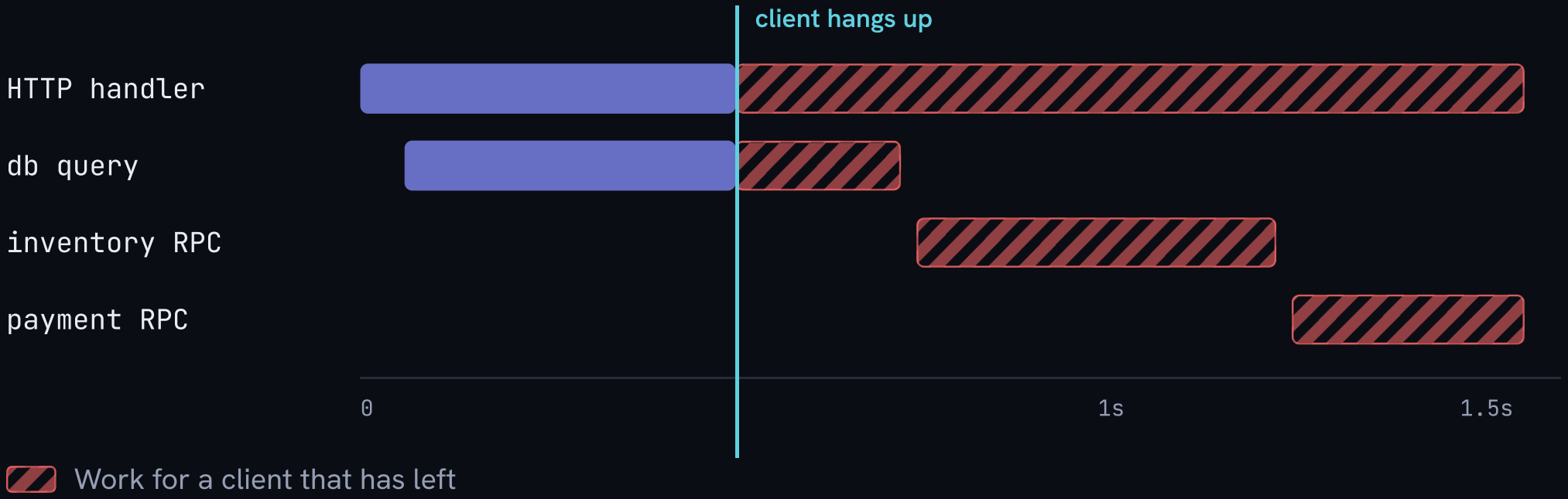


02

PART 2

Context primer

The work continues after the client leaves



The query holds a database connection the whole time

The inventory and payment services do work that nobody uses

CONTEXT PRIMER

context.Context

“

Package context defines the Context type, which carries deadlines, cancellation signals, and other request-scoped values across API boundaries and between processes.

context package doc

```
type Context interface {
    Deadline() (deadline time.Time, ok bool)   <- when to give up
    Done() <-chan struct{}                     <- closes when the work should stop
    Err() error                                <- Canceled or DeadlineExceeded
    Value(key any) any                         <- data about the request
}
```

Only the code that creates a context can cancel it

A context is read-only, so you add a deadline or a value with a child

A context ends in one of two ways

```
ctx, cancel := context.WithCancel(parent)
cancel()
ctx.Err() // context.Canceled
```

someone called cancel

```
ctx, cancel := context.WithTimeout(parent, 2*time.Second)
defer cancel()
<-ctx.Done()
ctx.Err() // context.DeadlineExceeded
```

the deadline passed

`Canceled` : the client disconnected, a shutdown signal arrived, or code called `cancel()`

`DeadlineExceeded` : a timeout ran out, in this context or in a parent

Each With function makes a child, so contexts form a tree

`Background()`

The empty root context

`WithCancel(parent)`

Stop when the code calls cancel

`WithTimeout(parent, d)`

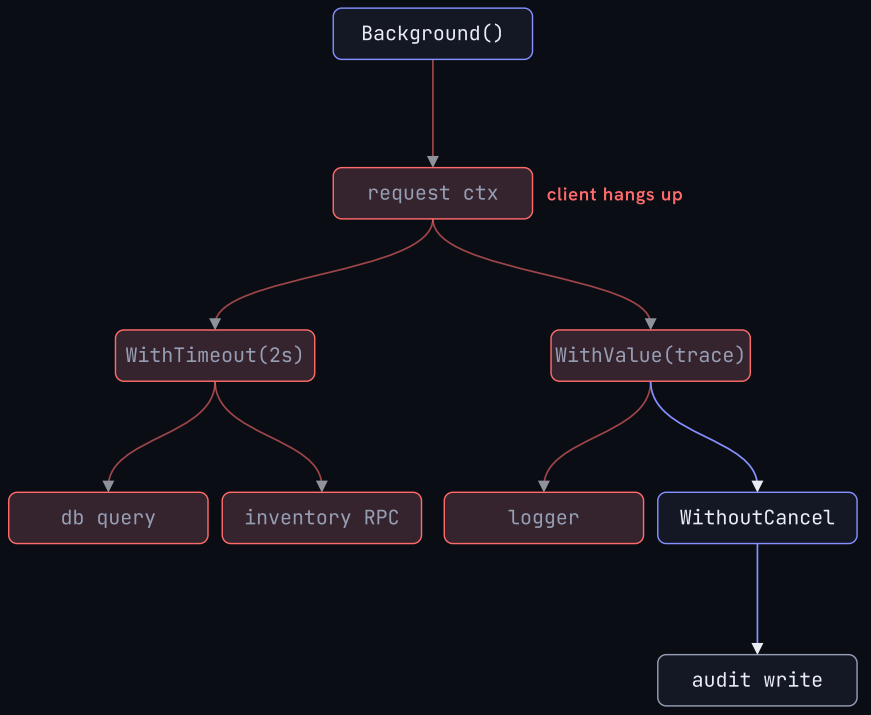
Stop after a duration. `WithDeadline` stops at a time

`WithValue(parent, k, v)`

Add one value

`WithoutCancel(parent)`

Keep the values and drop the stop signal (Go 1.21)



When the client hangs up, everything below the request stops. The WithoutCancel branch keeps going.

CONTEXT PRIMER

A context travels inside a process and between processes



In-process: your code passes `ctx` from function to function.
Distributed: gRPC and OpenTelemetry copy its deadline and trace into headers. They can copy only what your code passed down.

03

PART 3

Passing a context down

PASSING A CONTEXT DOWN

Use the caller's context

```
// Don't: start a new context here.
return s.store.Find(context.Background(), id)

// Do: pass the caller's context.
return s.store.Find(ctx, id)
```

loses the caller's deadline

Use `context.Background()` only in `main` and `init`

In tests, use `t.Context()` (Go 1.24)

Use `context.TODO()` where no context is passed in yet

Never pass a `nil` context

PASSING A CONTEXT DOWN

Pass ctx to each call, and don't store it in a struct

```go
// Don't: keep the context in a struct.
type Notifier struct {
    ctx    context.Context
    client *sms.Client
}


// Do: pass the context to each call.
func (n *Notifier) Send(ctx context.Context, msg string) error {
    return n.client.Send(ctx, msg)
}
```

one deadline for every call

> When you store the context in a struct, you obscure lifetime to the callers, or worse intermingle two scopes together in unpredictable ways.

Go blog, Contexts and structs, 2021

PASSING A CONTEXT DOWN

Wait on the timer and ctx.Done() together

```
// Don't: sleep without a check for cancel.
time.Sleep(backoff(attempt))

// Do: stop the wait when ctx is done.
select {
case <-time.After(backoff(attempt)):
case <-ctx.Done():
    return ctx.Err()
}
```

a cancel can't stop it

a cancel ends the wait

Every blocking wait needs a `ctx.Done()` case, including channel sends

In a loop, check `ctx.Err()` before each item

PASSING A CONTEXT DOWN

Give background work its own context and timeout

```go
// Don't: use the request context for background work.
go h.audit.Write(r.Context(), orderID) <-- canceled when the handler returns

// Do: keep the values and set a new timeout.
detached := context.WithoutCancel(r.Context()) <-- keeps the values (Go 1.21)
go func() {
    ctx, cancel := context.WithTimeout(detached, 5*time.Second) <-- its own timeout
    defer cancel()
    h.audit.Write(ctx, orderID)
}()
```

`context.Background()` drops the cancel too, but also the trace ID and log fields

04

PART 4

Giving every cancel a cause

GIVING EVERY CANCEL A CAUSE

Cancel with a cause that says why

```go
// Do: give each cancel a cause.
func placeOrder(ctx context.Context, id string) error {
    ctx, cancel := context.WithCancelCause(ctx)
    defer cancel(fmt.Errorf("placeOrder: completed: %w", context.Canceled))

    if err := checkInventory(ctx, id); err != nil {
        cancel(fmt.Errorf("inventory %s: %w: %w", id, err, context.Canceled))
        return err
    }
    // ...
}
```

Go 1.20

normal return

error path

```
$ context.Cause(ctx)
inventory ord-123: connection refused: context canceled
```

Only the first cancel sets the cause

`ctx.Err()` still returns `context.Canceled`

Work still running under `ctx` can read the cause

GIVING EVERY CANCEL A CAUSE

Wrap DeadlineExceeded in the cause, so errors.Is still matches

```
// Don't: use a cause that doesn't wrap DeadlineExceeded.
ctx, cancel := context.WithTimeoutCause(ctx, 20*time.Millisecond,
    errors.New("inventory: 20ms timeout"))
defer cancel()

_, err := http.DefaultClient.Do(req.WithContext(ctx))
errors.Is(err, context.DeadlineExceeded)


// Do: wrap DeadlineExceeded with %w.
ctx, cancel := context.WithTimeoutCause(ctx, 20*time.Millisecond,
    fmt.Errorf("inventory: 20ms timeout: %w", context.DeadlineExceeded))
```

errors.New doesn't wrap DeadlineExceeded

since Go 1.23, err is the cause

false

errors.Is now returns true

GIVING EVERY CANCEL A CAUSE

Stack a cancel cause on the timeout

```go
// Don't: rely on WithTimeoutCause alone.
ctx, cancel := context.WithTimeoutCause(ctx, 5*time.Second,
    fmt.Errorf("placeOrder %s: 5s timeout: %w", id, context.DeadlineExceeded))
defer cancel() // cancel() sets no cause


// Do: stack a cancel cause on the timeout.
ctx, cancelCause := context.WithCancelCause(ctx) // a cause on every other path
ctx, cancelTimeout := context.WithTimeoutCause(ctx, 5*time.Second, // the timer and the deadline
    fmt.Errorf("placeOrder %s: 5s timeout: %w", id, context.DeadlineExceeded))
defer cancelTimeout()
defer cancelCause(fmt.Errorf("placeOrder: completed: %w", context.Canceled)) // deferred last, so it runs first
```

GIVING EVERY CANCEL A CAUSE

When ctx ends, return its cause

```go
// Don't: return only what the driver says.
row := s.db.QueryRowContext(ctx, findOrderSQL, id)
if err := row.Scan(&o.ID, &o.Status); err != nil {
    return Order{}, fmt.Errorf("find order: %w", err)
}


// Do: when ctx has ended, return its cause.
if err := row.Scan(&o.ID, &o.Status); err != nil {
    if ctx.Err() != nil {
        err = context.Cause(ctx)
    }
    return Order{}, fmt.Errorf("find order: %w", err)
}
```

says what ended, not why

the cause that placeOrder set

errors.Is still matches

In a `select` on `ctx.Done()`, return `context.Cause(ctx)` the same way

With no cause set, `context.Cause(ctx)` returns `ctx.Err()`

Log the error once, in the handler

```go
// Don't: log in the store and return the error too.
slog.ErrorContext(ctx, "find order failed", "err", err)
return Order{}, fmt.Errorf("find order: %w", err)


// Do: log once, in the handler.
if err := placeOrder(r.Context(), r.PathValue("id")); err != nil {
    slog.ErrorContext(r.Context(), "place order", "err", err)
    http.Error(w, "order failed", http.StatusInternalServerError)
    return
}
```

one more line for the same failure

```
level=ERROR msg="place order"
    err="find order: placeOrder ord-123: 5s timeout: context deadline exceeded"
```

05

PART 5

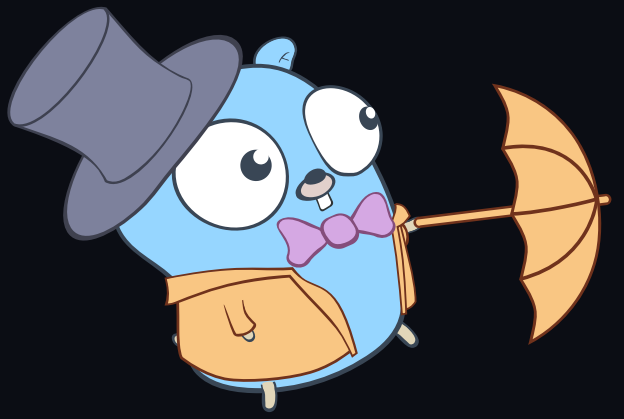
Context values

CONTEXT VALUES

Put a value in the context only if the code works without it

PUT IN A CONTEXT

- Trace, request, and correlation IDs
- Auth tokens from middleware
- Request-scoped log fields
- Idempotency keys



MAKE IT A PARAMETER

- User data the function works on
- Database handles and service clients
- Configuration
- Business logic inputs

> Use context Values only for request-scoped data that transits processes and APIs, not for passing optional parameters to functions.

context package doc

CONTEXT VALUES

Hide an unexported key behind a setter and a getter

```
// Don't: use a string as the key.
ctx = context.WithValue(ctx, "user", "alice")     // package auth
ctx = context.WithValue(ctx, "user", "audit-bot") // package audit
ctx.Value("user")                                 // "audit-bot"
```

audit-bot hides alice

```
// Do: keep an unexported key behind a setter and a getter.
package auth

type ctxKey struct{}

func WithUser(ctx context.Context, u User) context.Context {
    return context.WithValue(ctx, ctxKey{}, u)
}

func UserFromContext(ctx context.Context) (User, bool) {
    u, ok := ctx.Value(ctxKey{}).(User)
    return u, ok
}
```

only auth can make one

a typed value and a bool

06

PART 6

Enforcing the rules

ENFORCING THE RULES

Linters can check most of these rules

`go vet: lostcancel`	A cancel function that is never called
`staticcheck: SA1012`	A `nil` context passed to a function
`staticcheck: SA1029`	A built-in type, such as a string, used as a key
`containedctx`	A context stored in a struct field
`contextcheck`	A function that uses a context it didn't get from its caller
`errorlint`	`err == context.Canceled`, which is false when a cause wraps it

ENFORCING THE RULES

Wrap the constructors so every cancel needs a cause

```go
// Do: make the cancel func take a cause.
package xctx

func WithCancel(parent context.Context) (context.Context, context.CancelCauseFunc) {
    ctx, cancel := context.WithCancelCause(parent) //nolint:forbidigo
    return ctx, func(cause error) {   // <- the cancel func needs a cause
        if cause == nil {             // <- nil still gets a cause
            cause = errors.New("no cause given")
        }
        cancel(fmt.Errorf("%w: %w", cause, context.Canceled))  // <- errors.Is still matches
    }
}
```

```yaml
# Do: block the plain constructors in .golangci.yml.
version: "2"
linters:
  enable: [forbidigo]
  settings:
    forbidigo:
      analyze-types: true
      forbid:
        - pattern: ^context\.With(Cancel|Timeout|Deadline)(Cause)?$
```

At DoorDash, Alex Hornbake built this as `xctx` for a Go monorepo that many teams share.

WRAP-UP

The checklist I use in code review

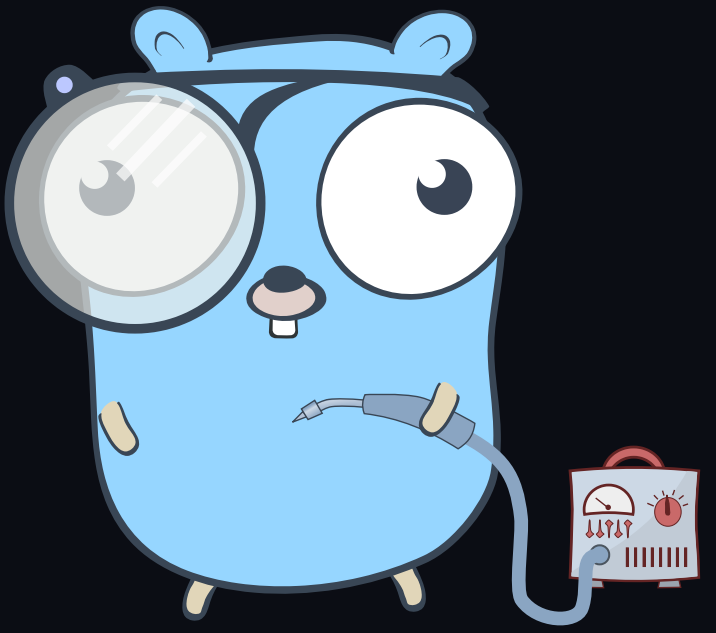
1. Pass `ctx` to each call, and don't store it in a struct
2. Check `ctx` in every loop and every blocking wait
3. Give background work `WithoutCancel` and a timeout
4. Give every cancel a cause, including the normal return
5. Stack `WithCancelCause` on a timeout to cover every path
6. Wrap `Canceled` or `DeadlineExceeded` in every cause
7. Return `context.Cause(ctx)`, and log once in the handler
8. Put only optional data in a context, and hide the key



WRAP-UP

Some cancels still have no cause

- The `net/http` server, when the client hangs up
- Libraries that call a plain `cancel()`
- Other services, because gRPC sends only the deadline and the cancel signal



REFERENCES

Further reading

GO BLOG

- Go Concurrency Patterns: Context, Sameer Ajmani, 2014
- Go Concurrency Patterns: Pipelines and cancellation, Sameer Ajmani, 2014
- Contexts and structs, Jean Barkhuysen and Matt T. Proud, 2021

MY POSTS

- What canceled my Go context?
- Avoiding collisions in Go context keys
- What belongs in Go's context values?
- Is passing user ID through context an antipattern?
- Stacked log lines considered harmful

GO ISSUES

- #26356: context: ease debugging of where a context was canceled?
- #40221: context: add WithoutCancel
- #51365: context: add APIs for writing and reading cancelation cause
- #56661: context: add APIs for setting a cancelation cause when deadline or timer expires

DOCS AND STYLE GUIDES

- Package context
- Go 1.20 release notes: context
- Go 1.21 release notes: context
- Google Go style guide: Contexts
- Google Go style guide: Custom contexts

LINTERS

- go vet: lostcancel
- staticcheck: SA1012, SA1029
- golangci-lint: containedctx, contextcheck, errorlint, forbidigo

Thanks

Questions?

REDNAFI.COM

